Database Management Systems: Unit 3 Lecture Notes

Subject: Database Management Systems (DBMS) | Unit 3: Transaction Processing

1. Introduction to Database Transactions

A transaction is a logical unit of work in a database management system that may contain one or more database operations such as read, insert, update, or delete. A transaction must either execute completely or have no effect at all. This all-or-nothing guarantee is fundamental for maintaining database consistency and data reliability. Every transaction transitions through several distinct states: Active, Partially Committed, Failed, Aborted, and Committed.

2. The ACID Properties of Transactions

To ensure integrity during concurrent executions and system failures, transactions must strictly adhere to the ACID properties: Atomicity guarantees that all operations within the work unit execute completely or none do, handled primarily by the undo log mechanism. Consistency ensures that a transaction takes the database from one valid state satisfying all integrity constraints to another valid state. Isolation ensures that concurrent transactions execute independently without interfering with each other's intermediate uncommitted states. Durability guarantees that once a transaction commits, its modifications persist permanently on non-volatile disk storage even in the event of an abrupt system power crash.

3. Concurrency Control Mechanisms

When multiple transactions access shared data items simultaneously, anomalies like dirty reads, lost updates, and unrepeatable reads can occur. Concurrency control protocols ensure serializability so that the outcome of interleaved operations is equivalent to some purely sequential execution order. The most widely adopted protocol is Two-Phase Locking (2PL), which consists of two distinct phases: the Growing Phase and the Shrinking Phase.

In the Growing Phase of 2PL, a transaction acquires locks (Shared or Exclusive) as needed and cannot release any locks. Once a transaction releases any lock, it enters the Shrinking Phase, where it can only release locks and is prohibited from requesting any new locks. Strict Two-Phase Locking (Strict 2PL) additionally requires that all exclusive locks must be held until the transaction explicitly commits or aborts, preventing cascading rollbacks.

4. Deadlock Management in DBMS

A deadlock occurs when two or more transactions wait indefinitely for locks held by each other in a cyclic dependency. Deadlocks can be detected using a Wait-For Graph (WFG), where vertices represent transactions and directed edges represent lock requests. A cycle in the Wait-For Graph indicates a deadlock. Common resolution techniques include victim selection based on transaction age, cost of rollback, and lock timeouts.

5. Indexing Architectures and B+ Tree Fundamentals

Database indexing significantly accelerates query retrieval speeds without requiring full table scans. A B+ Tree is a self-balancing, multi-way search tree data structure optimized for block-based disk storage systems. In a B+ Tree, all data records or record pointers reside exclusively in the leaf nodes, while internal nodes store only routing keys and child pointers.

Leaf nodes in a B+ Tree are linked together sequentially in a doubly-linked list. This critical structural design makes B+ trees exceptionally efficient for range queries and ordered scans. The search time complexity in a B+ tree is O(log_B N), where B is the branching factor (order) and N is the number of keys. Because disk block input/output operations are costly, B+ trees are configured with high fan-out so the tree height typically remains shallow (around 3 to 4 levels) even for billions of records.